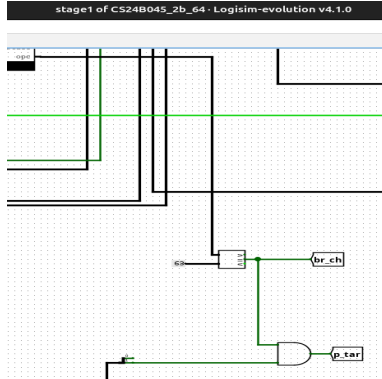
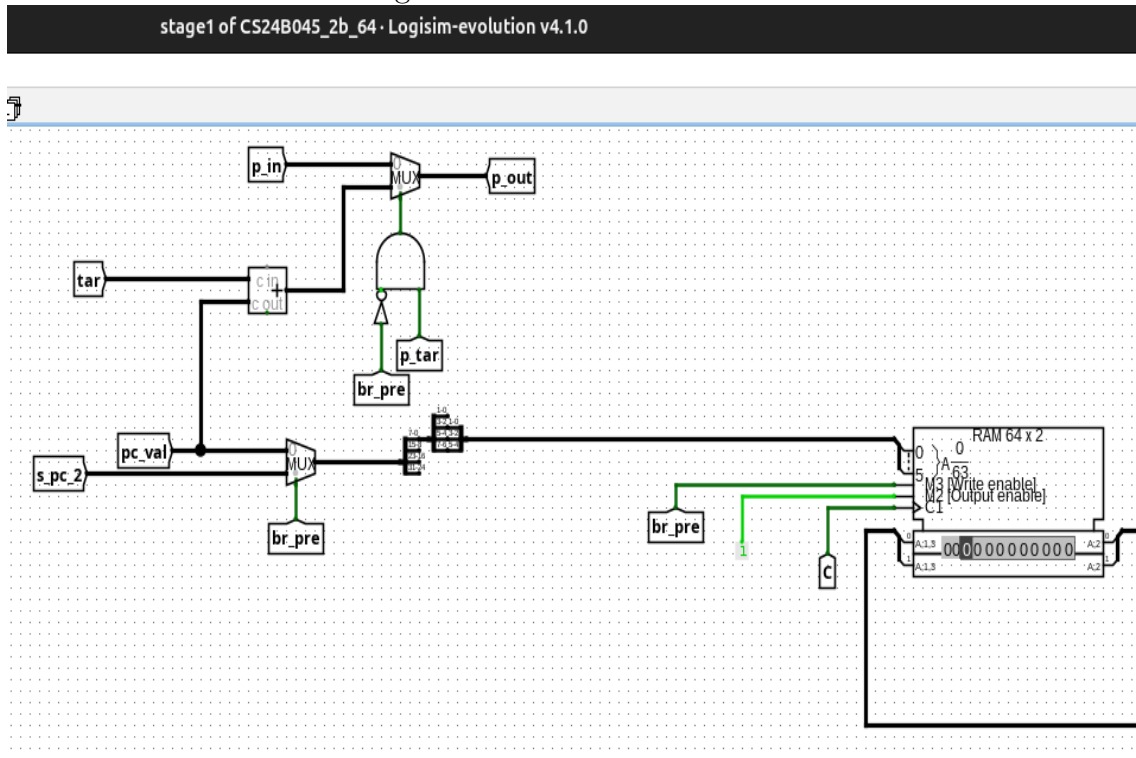


CS24B045 | Lab 11 Report

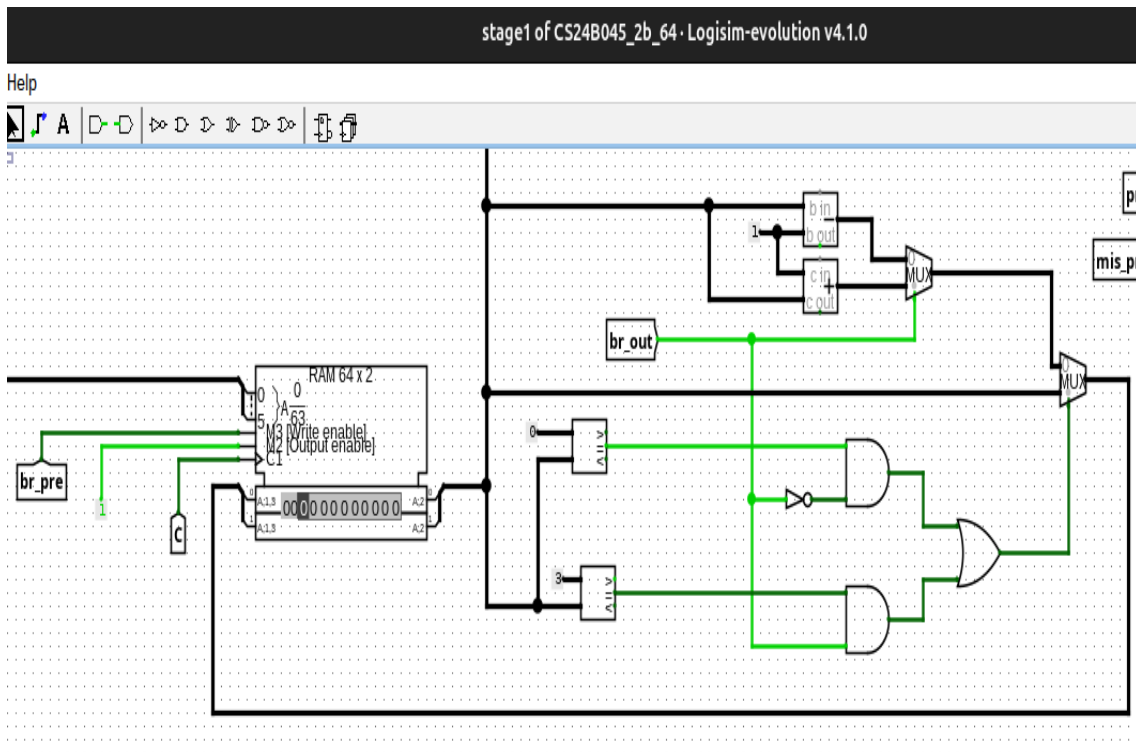
Description of the branch Predictor implementation:



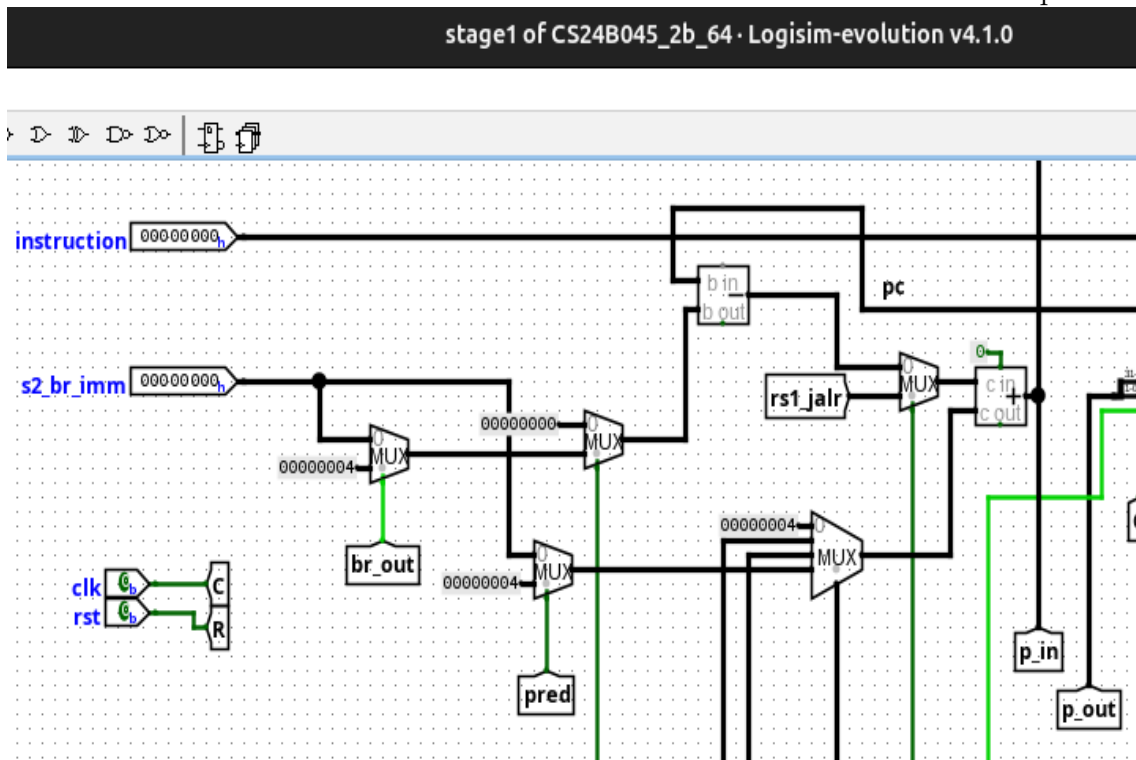
- Here br_ch tells that if the present instruction in the S1 is b-type or not.
- If the both the conditions that the present ins in S1 is b-type and also the MSB of the counter is 1 then the target select bit is set to 1.



- If the previous one is not branch and the present one is branch and it can take target by the prediction then pc value is incremented by the target value.
- Also if the previous one is of the b-type then we need to update the value of the counter for that pc so that the write enable of the RAM is only if the previous one is branch(i.e, the instruction in the S2).



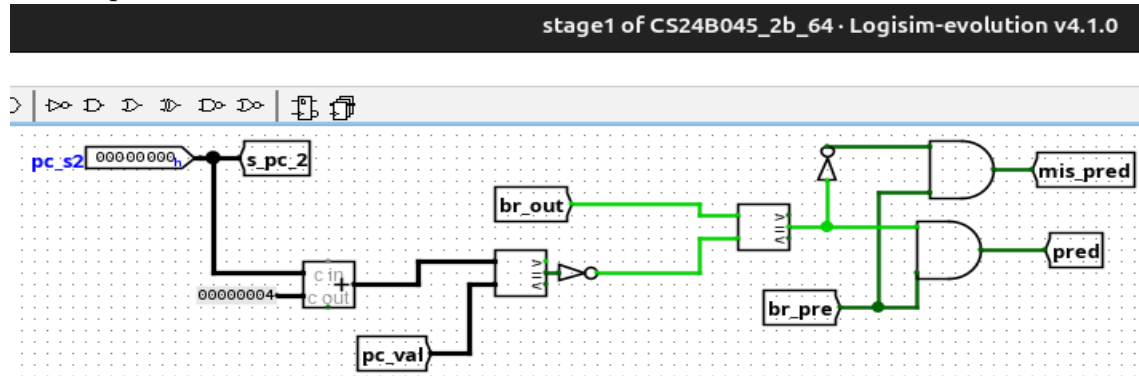
- If the counter of that pc value is 3 and in the previous step also the branch is taken then the value of the cannot increase more similarly if the counter value is 0 and the branch is not taken in the previous step then the value of the counter cannot decrease than 0 so in both the cases the value of the counter remains same.
- In the other cases the value of the counter is increased or decreased as required.



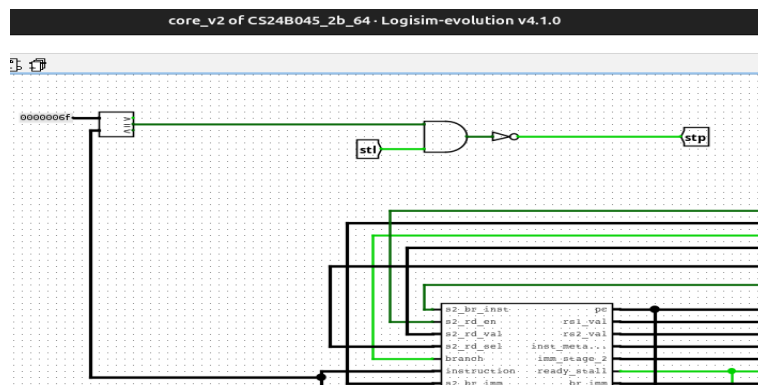
- If the previous instr. is a branch(i.e, the instr. at present in S2) and the prediction is correct then the 4-way MUX selects the address 3 value pin i.e, s2_br_imm but as it is

correct we only need to do +4 so added that 2-way-MUX that if the prediction is correct then it gives 4.

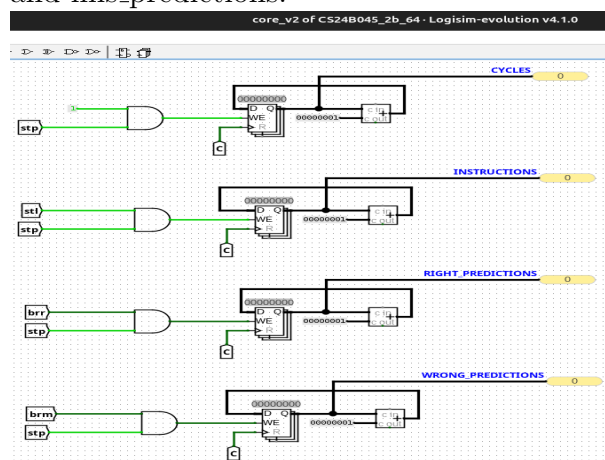
- Also if the prediction is wrong then the pc value is restored by decrementing the value by 4 or s2.br_imm according to the previously added value to the pc and through the 4-way-MUX the target value is added to the pc also there will be stall introduced at this step.



- To count the number of the mis_predictions we take the tunnel mis_pred with condition that previous is branch and also it mispredicts only then the value of the mis_pred is 1.



- If the instruction is 0x6f and there is no stall then it will execute in the S2 and so the program is completed and so we can stop counting the instruction, cycles, predictions and mis_predictions.

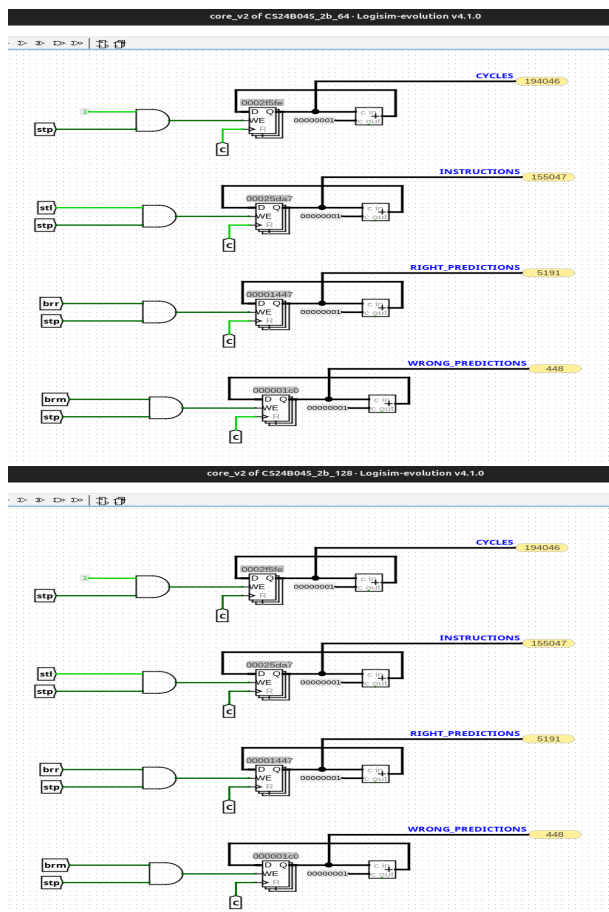


- The cycles increment by 1 in every cycle performed till the last instruction is executed
- and for the number of the instructions the instructions will be incremented if that instruction is executed in the S2 also (by using the stall wire and logic)

- the number of right predictions is incremented only when the instruction(in S2) is branch and it is predicted correct in the S1(i.e, the next instruction of the b-type instruction)
- similarly the number of the wrong predictions is incremented only when the instruction in the S2 is b-type and also the predicted value is wrong in S1.

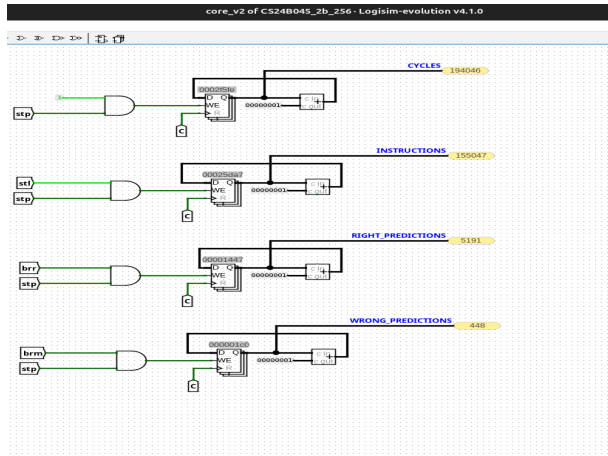
Bonus1:

- As the number of the entries of the branch predictor increases then the number of the bits to be extracted from pc also increases and so the collision of the same entry for different pcs will decrease.
- Below are the numbers for the mcu_assign_mul.



This is for 2-bit 64 entries one.

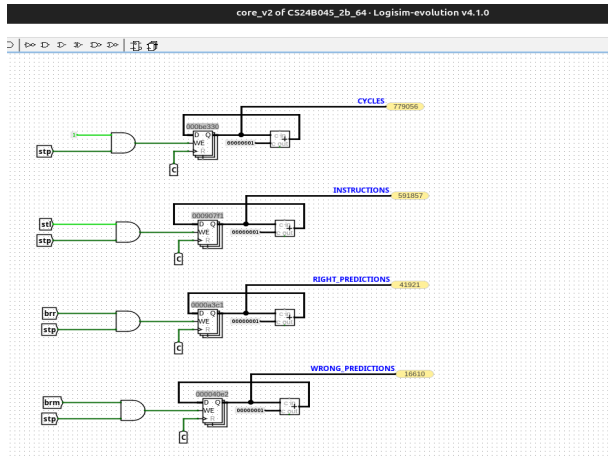
This is for 2-bit 128 entries one.



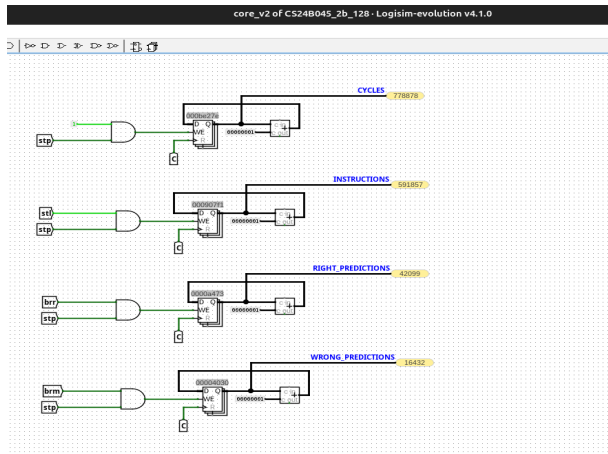
This is for 2-bit 256 entries one.

- As from the above values we can tell that the collisions on the same last 6,7,8 bits of the pc for the b-type instructions in null for the mcu_assign_mul testcase.

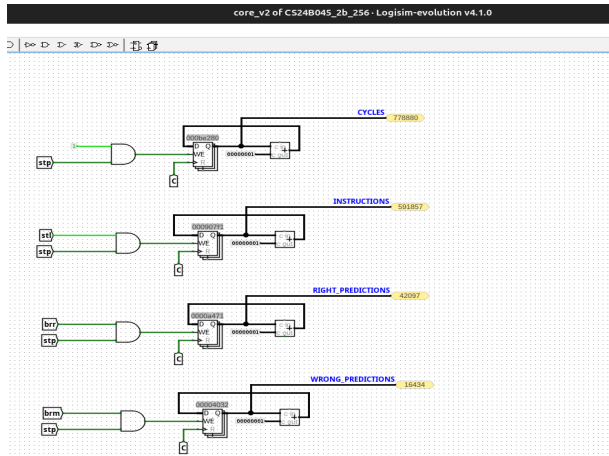
- Below are the numbers for the mcu_assign_nomul.



This is for 2-bit 64 entries one.



This is for 2-bit 128 entries one.



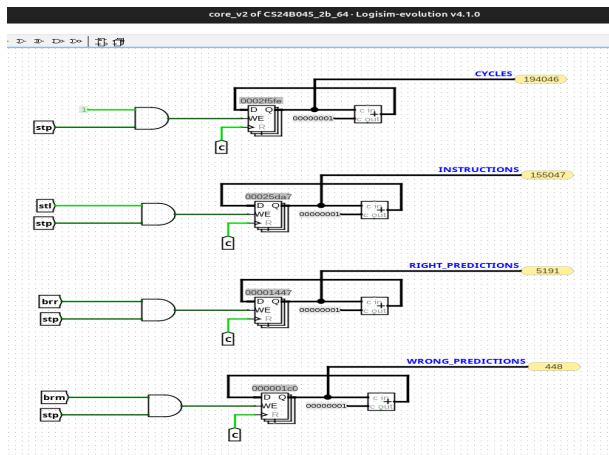
This is for 2-bit 256 entries one.

- As from the above values we can tell that the collisions on the same last 6 and for last (7,8) bits got decreased, by the values from the right_predictions and the wrong_predictions we can tell that for the mcu_assign_nomul testcase.
- So for 64 entries to 128 there is some improvement and from the 128 to 256 entries there is no change in this case.

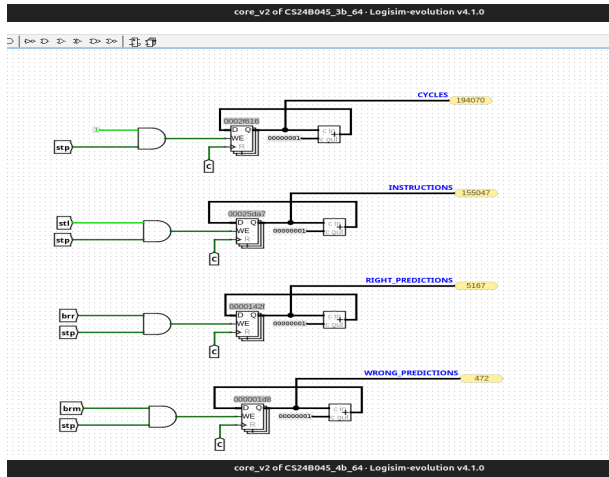
Bonus2:

- As the size of the counter increases it will take more turns to give the correct prediction in most cases like if we take an example of the for loop for the half of the counter max_val times it gives wrong prediction at start and as always at end also but there is more chance for the deprovement instead of improvement.
- But it will vary code to code that in some code may be less bit is more efficient where as in some other code more bit one is more efficient.

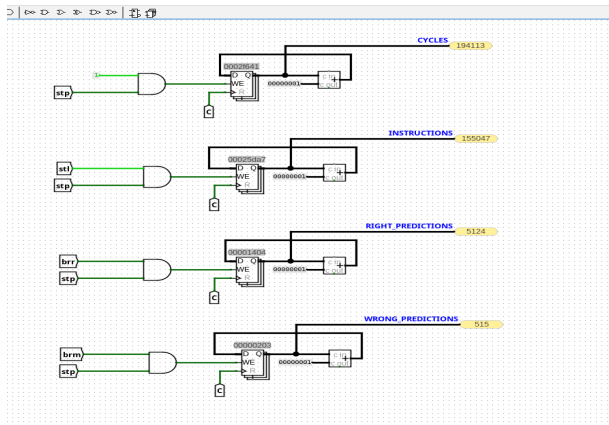
- Below are the numbers for the mcu_assign_mul.



This is for 2-bit 64 entries one.



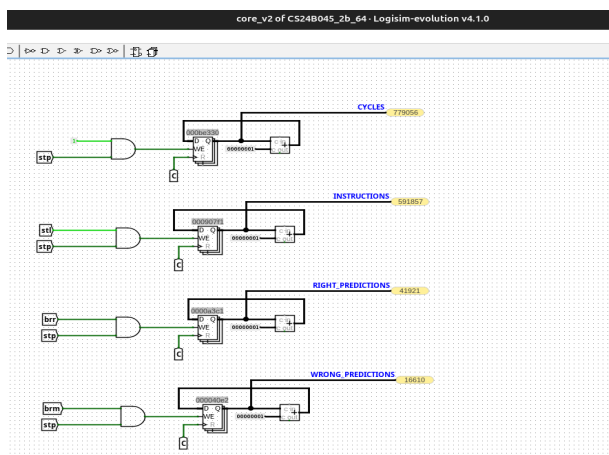
This is for 3-bit 64 entries one.



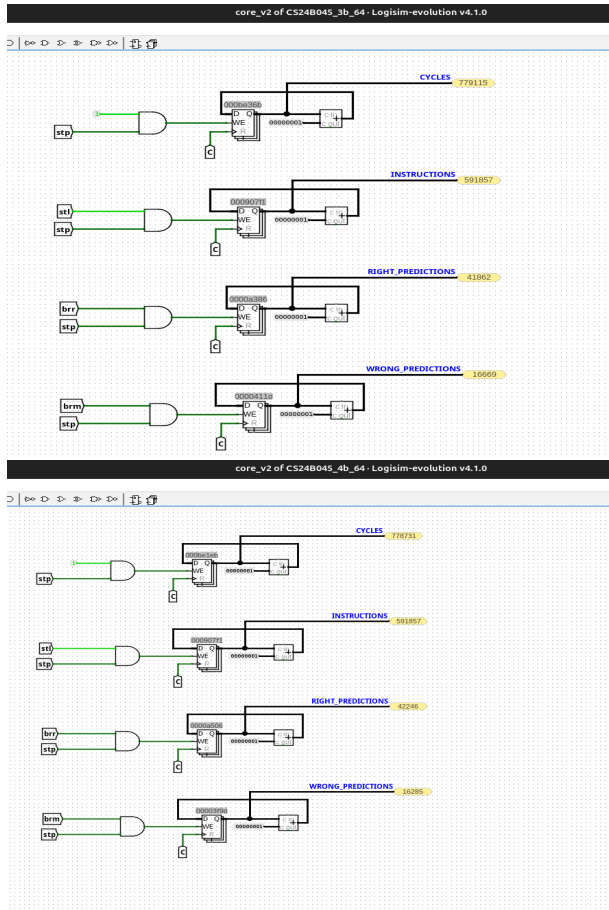
This is for 4-bit 64 entries one.

- As from the above values we can tell that the number of the right predictions is decreased and the wrong predictions increase which lead to the less efficient one for the mcu_assign_mul testcase.

- Below are the numbers for the mcu_assign_nomul.



This is for 2-bit 64 entries one.



This is for 3-bit 64 entries one.

This is for 4-bit 64 entries one.

- As from the above values we can tell that the number of the right predictions is decreases from the 2-bit to 3-bit counter and also increased from the 3-bit to 4-bit counter in this case of mcu_assign_nomul testcase.